

HDOC Day-35

Square Matrix Diagonal Operations Using Separate Functions

Aim

To write a menu-driven C program that reads a variable-length square integer array and uses separate functions to check whether all main-diagonal elements are distinct, calculate their sum, and display the matrix using the required alternating diagonal traversal.

Algorithm

1. Start the program and read the order n of the square matrix.
2. Declare a variable-length array $A[n][n]$ and read all matrix elements.
3. Display a menu with: (1) check distinct main-diagonal elements, (2) sum main-diagonal elements, (3) diagonal traversal, and (4) Exit.
4. For option 1, compare every main-diagonal element $A[i][i]$ with the remaining main-diagonal elements. If no duplicate is found, report that all are distinct.
5. For option 2, add $A[i][i]$ for $i = 0$ to $n-1$ and display the sum.
6. For option 3, process diagonals in increasing order of row+column index. Traverse successive diagonals alternately upward-right and downward-left, beginning at $A[0][0]$.
7. For an invalid menu choice, display an error message. Continue until Exit is selected.
8. Stop the program.

C Program

```
#include <stdio.h>

void checkDistinct(int n, int a[n][n]);
void diagonalSum(int n, int a[n][n]);
void diagonalTraversal(int n, int a[n][n]);

int main()
{
    int n, i, j, choice;

    printf("Enter order of square matrix: ");
    scanf("%d", &n);

    int a[n][n];

    printf("Enter matrix elements:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    do
    {
        printf("\n----- MATRIX DIAGONAL MENU -----\n");
        printf("1. Check distinct main-diagonal elements\n");
        printf("2. Sum of main-diagonal elements\n");
        printf("3. Diagonal traversal\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: checkDistinct(n, a); break;
```

```

        case 2: diagonalSum(n, a); break;
        case 3: diagonalTraversal(n, a); break;
        case 4: printf("Exiting program...\n"); break;
        default: printf("Invalid choice! Please enter 1 to 4.\n");
    }
} while (choice != 4);

return 0;
}

void checkDistinct(int n, int a[n][n])
{
    int i, j, distinct = 1;

    printf("Main diagonal elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i][i]);
    printf("\n");

    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[i][i] == a[j][j])
                distinct = 0;

    if (distinct)
        printf("All the main diagonal elements are distinct.\n");
    else
        printf("The main diagonal elements are not all distinct.\n");
}

void diagonalSum(int n, int a[n][n])
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i][i];
    printf("Sum of main diagonal elements: %d\n", sum);
}

void diagonalTraversal(int n, int a[n][n])
{
    int d, r, c;
    printf("Diagonal traversal: ");

    for (d = 0; d <= 2 * n - 2; d++)
    {
        if (d % 2 == 0)
        {
            r = (d < n) ? d : n - 1;
            c = d - r;
            while (r >= 0 && c < n)
            {
                printf("%d ", a[r][c]);
                r--; c++;
            }
        }
        else
        {
            c = (d < n) ? d : n - 1;
            r = d - c;
            while (c >= 0 && r < n)
            {
                printf("%d ", a[r][c]);
                r++; c--;
            }
        }
    }
    printf("\n");
}

```

Test Case Table

Test	Operation	Input / Choice	Expected Result
1	Distinct diagonal	3x3: 1 2 3 / 4 5 6 / 7 8 9	Diagonal: 1 5 9; all distinct

Test	Operation	Input / Choice	Expected Result
2	Diagonal sum	Same 3x3 matrix	Sum = 15
3	Diagonal traversal	Same 3x3 matrix	1 2 4 7 5 3 6 8 9
4	Not distinct	3x3: 1 2 3 / 4 1 6 / 7 8 9	Diagonal 1 1 9; not all distinct
5	Invalid menu	Choice = 5	Invalid choice message
6	Exit	Choice = 4	Program exits

Result

The menu-driven C program was successfully implemented using a variable-length square 2D array and separate functions. It checks whether the main-diagonal elements are distinct, calculates the sum of the main diagonal, and performs the required alternating diagonal traversal. For the example matrix containing 1 to 9, the traversal is 1 2 4 7 5 3 6 8 9 and the main-diagonal sum is 15.